

Option C Task Report

COS30018 – Intelligent Systems

Lecturer: Dr. Nguyen Manh Toan

Group: 3.1 – OpenBuffer

Task name: **C.2 – Data Processing 1**



We've implemented the **load_and_process_dataset** function that satisfies all the functional requirements from the task description. The final objective of this function is to allow user flexibility in choosing the timeframe of the stock's dataset.

Here is the breakdown of what we've done for each requirement:

Fuction's Params:

```
def load_and_process_dataset(  
    company: str,  
    start_date: str,  
    end_date: str,  
    features: list | None = None,  
    data_dir: str = 'data',  
    force_download: bool = False,  
    nan_method: str = 'forward_fill',  
    split_method: str = 'date',  
    split_param: str = '2023-01-01',  
    scale_columns: bool = False  
):
```

- **company (type: str):** Represents the stock ticker or market symbol of the target corporation.
- **start_date & end_date (type: str):** Define the absolute bounds of the historical timeline, formatted as 'YYYY-MM-DD' strings.
- **features (type: list | None, default: None):** Accepts a specific list of market data columns to extract from the raw dataset, such as ['Open', 'High', 'Low', 'Close', 'Volume'].
- **data_dir (type: str, default: 'data'):** Specifies the target directory on your local machine where downloaded stock datasets are managed.
- **force_download (type: bool, default: False):** A Boolean switch that forces the system to pull a fresh copy of the dataset from the web, ignoring any existing files on disk.
- **nan_method (type: str, default: 'forward_fill'):** Determines the mathematical approach used to clean missing data values (NaN) from the time-series arrays.
- **split_method (type: str, default: 'date'):** Controls the core architectural approach used to split the dataset into your training and testing matrices.
- **split_param (type: str, default: '2023-01-01'):** Provides the explicit modifier value or threshold rule utilized by your chosen split_method.
- **scale_columns (type: bool, default: False):** A Boolean toggle that decides whether or not to map feature distributions down to normalized ranges like $[0, 1]$ before exiting the function.

1. Requirement a - Flexible Date Specification and Validation:

The function uses *pd.to_datetime* to convert the input date strings into formal Pandas datetime objects. It introduces a vital logical sanity check with a conditional statement to ensure the start date precedes the end date chronologically, preventing invalid input like the start date and the end are the same day.

```
1 # [REQUIREMENT A] Specify start date and end date for the whole dataset
2 start_dt = pd.to_datetime(start_date)
3 end_dt = pd.to_datetime(end_date)
4 if start_dt >= end_dt:
5     raise ValueError("Start Date must be before the End Date.")
```

2. Requirement b - NaN Management:

The function is built to handle financial data missingness caused by weekends and market holidays. It supports **data imputation** techniques: **forward filling (ffill)**, **backward filling (bfill)**, and **column-mean**. The inclusion of a trailing *bfill()* statement serves as a safety fallback to clear any NaN values residing at the very beginning of the dataset where forward filling is ineffective.

```
1 # [REQUIREMENT B] Deal with the NaN issue in the data
2 # forward_fill: take the last value to fill (e.g. on Friday)
3 # backward_fill: take the first value from the future to fill -> data leakage
4 if nan_method == 'forward_fill':
5     df = df.ffmpeg()
6 elif nan_method == 'drop':
7     df = df.dropna()
8 elif nan_method == 'mean':
9     df = df.fillna(df.mean())
10 df = df.bfill() # Safety fallback
```

3. Requirement c - Dynamic Data Splitting Methods:

The function supports different dataset splitting strategies determined by user-defined arguments. When set to split by **date**, it utilizes a timestamp cutoff to partition data chronologically without shuffling, preserving time-series consistency and preventing data leakage. It also features a **ratio-based** approach that segments the data sequentially using a specified fractional threshold (e.g. 80/20 split).

```

1  # [REQUIREMENT C] Use different methods to split the data into train/test data
2  # based on split-method and split-param arguments
3  if split_method == 'date':                                # Split with date
4      cutoff = pd.Timestamp(split_param)
5      train_df = df[df.index < cutoff].copy()
6      test_df = df[df.index >= cutoff].copy()
7  elif split_method == 'ratio':                             # Split with ratio
8      split_idx = int(len(df) * float(split_param))
9      train_df = df.iloc[:split_idx].copy()
10     test_df = df.iloc[split_idx:].copy()
11 else:
12     train_df = df.copy()
13     test_df = pd.DataFrame()

```

4. Requirement d - Local Data Caching Framework:

Redundant download segments from the original script have been completely eliminated. The function checks for the historical existence of the target dataset locally using *os.path.exists* inside a dedicated directory. If found and not overridden by the force download flag, it reads the local CSV file instantly to save time and bandwidth; otherwise, it downloads the fresh data via Yahoo Finance.

```

1  # [REQUIREMENT D] Store downloaded data locally and load locally to save time
2  os.makedirs(data_dir, exist_ok=True)
3  data_file = os.path.join(data_dir, f'{company}_{start_date}_{end_date}.csv')
4

```

5. Requirement e - Multi-Feature Scaling and Scaler Storage:

The function includes an integrated scaling pipeline that iterates over all specified feature columns. It instantiates a separate *MinMaxScaler* for each feature column, performs a *fit_transform* exclusively on the training set to avoid data leakage, applies a transform on the test set, and stores every fitted scaler inside a dictionary structure for downstream *inverse-transform* operations.

```
1 # [REQUIREMENT E] Option to scale feature columns and store scalers in a data structure
2 scalers = {}
3 if scale_columns:
4     for col in train_df.columns:
5         scaler = MinMaxScaler(feature_range=(0, 1))
6         train_df[col] = scaler.fit_transform(train_df[[col]])
7         if len(test_df) > 0:
8             test_df[col] = scaler.transform(test_df[[col]])
9         scalers[col] = scaler
10
11 return train_df, test_df, scalers
```